Question 4 - Centralized logging with the ELK stack

Question 4: Implement centralized logging using the ELK stack and demonstrate log collection, parsing, and visualization in Kibana.

All commands below were run on the lab machine and their output is copied verbatim from the recorded logs in `evidence/logs/`. Repository: https://github.com/ksamirdev/ensp461-devops-lab3.

Log format

The app writes one line per request to `/var/log/text-stats/app.log` on a Docker volume. Logstash mounts the same volume read-only. Requests that end in 4xx or 5xx are logged at WARNING.

service/web.py (log line)

```
level = logging.WARNING if response.status_code >= 400 else logging.INFO
    log.log(
        level,
        "method=%s path=%s status=%s duration_ms=%.2f",
        request.method, route, response.status_code, elapsed * 1000,
    )
```

Logstash pipeline

observability/logstash/pipeline.conf

```
input {
  file {
    path => "/var/log/text-stats/app.log"
    start_position => "beginning"
    sincedb_path => "/dev/null"
  }
}

filter {
  grok {
    match => {
      "message" => "%{TIMESTAMP_ISO8601:logged_at} %{LOGLEVEL:level} %{WORD:logger} method=%{WORD:me
thod} path=%{NOTSPACE:route} status=%{NUMBER:status:int} duration_ms=%{NUMBER:duration_ms:float}"
    }
  }
  date {
    match => ["logged_at", "yyyy-MM-dd HH:mm:ss,SSS"]
    timezone => "UTC"
    remove_field => ["logged_at"]
  }
}

output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    index => "text-stats-logs"
  }
}
```

Grok splits each line into typed fields (`status` as an integer, `duration_ms` as a float). The date filter replaces the ingest time with the time written in the log line.

Deploy the stack

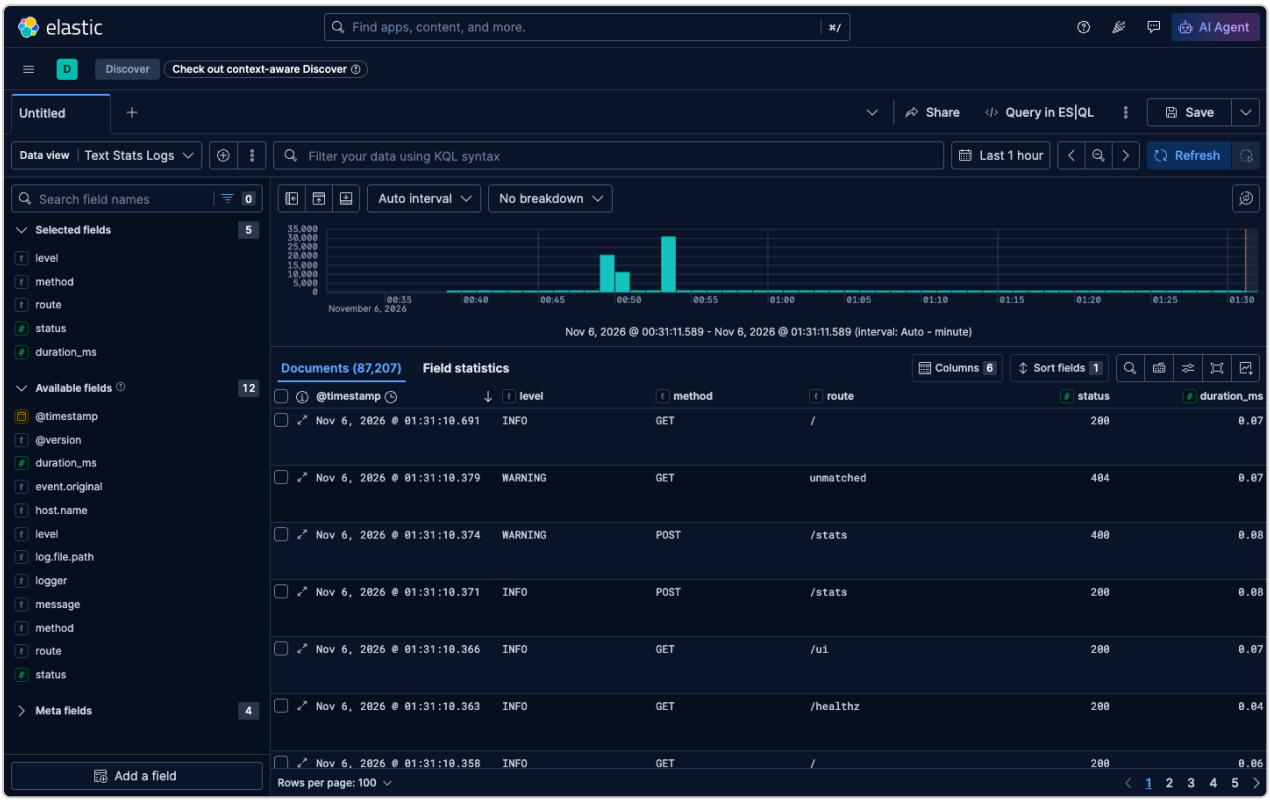
```
$ docker compose -f observability/compose.yml up -d elasticsearch logstash kibana 2>&1 | grep -E 'St
arted|Running|Healthy'
 Container lab3-observability-elasticsearch-1 Started
 Container lab3-observability-elasticsearch-1 Healthy
 Container lab3-observability-logstash-1 Started
 Container lab3-observability-kibana-1 Started
$ until curl -sf http://localhost:9290/_cluster/health >/dev/null; do sleep 3; done; curl -s http://
localhost:9290/_cluster/health | jq -c '{cluster_name,status,number_of_nodes}'
{"cluster_name":"docker-cluster","status":"green","number_of_nodes":1}
$ docker compose -f observability/compose.yml ps --format '{{.Service}}  {{.Status}}'
app  Up 43 minutes (healthy)
elasticsearch  Up 21 seconds (healthy)
grafana  Up 43 minutes
kibana  Up Less than a second
logstash  Up Less than a second
prometheus  Up 43 minutes
```

Collection and parsing

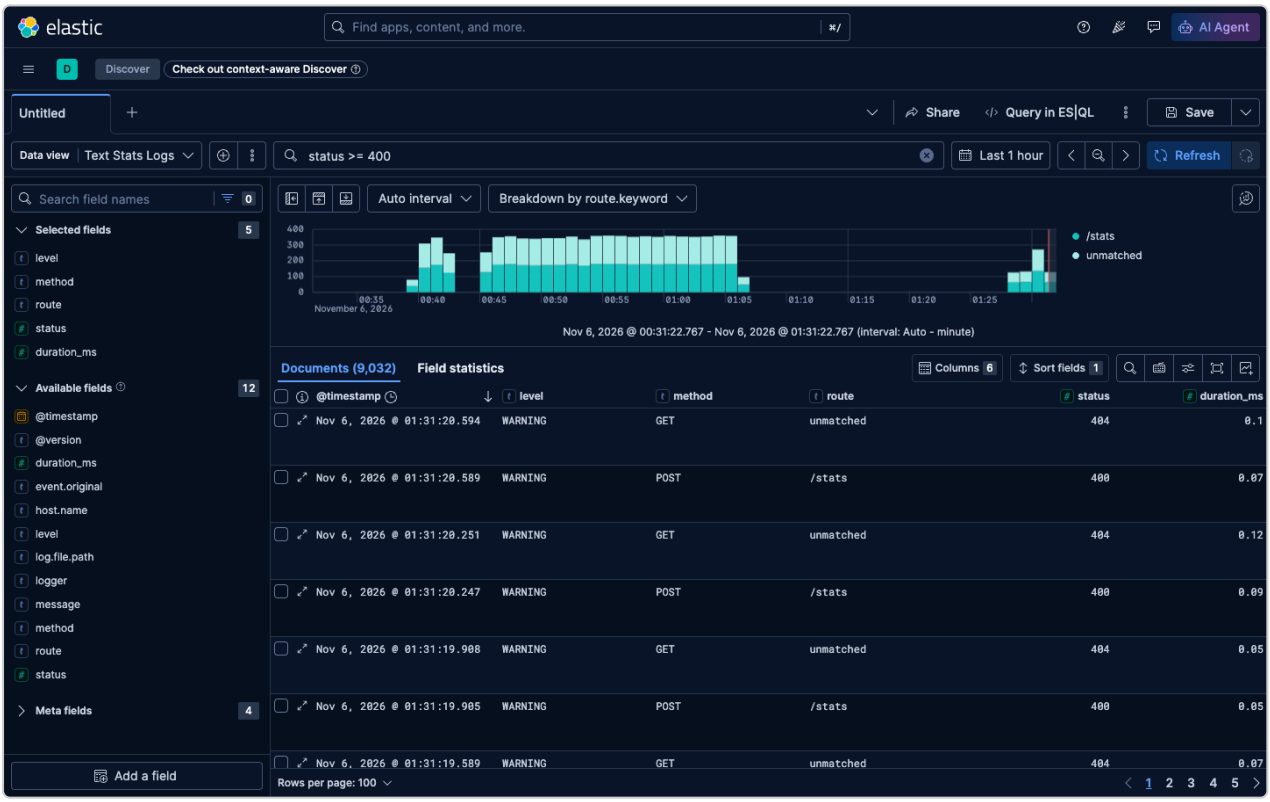
```
$ docker compose -f observability/compose.yml exec -T app tail -3 /var/log/text-stats/app.log
2026-11-05 19:59:23,022 WARNING textstats method=GET path=unmatched status=404 duration_ms=0.09
2026-11-05 19:59:28,318 INFO textstats method=GET path=/healthz status=200 duration_ms=0.05
2026-11-05 19:59:38,405 INFO textstats method=GET path=/healthz status=200 duration_ms=0.38
$ curl -s http://localhost:9290/_cat/indices/text-stats-logs?v
health status index           uuid                   pri rep docs.count docs.deleted store.size pri.
store.size dataset.size
yellow open   text-stats-logs hrCh5MfWSFSsmI7-tj_W-A   1   1      86072            0       10mb
10mb         10mb
$ curl -s 'http://localhost:9290/text-stats-logs/_search?size=1&sort=@timestamp:desc' | jq '.hits.hi
ts[0]._source | {"@timestamp",level,logger,method,route,status,duration_ms}'
{
  "@timestamp": "2026-11-05T19:59:38.405Z",
  "level": "INFO",
  "logger": "textstats",
  "method": "GET",
  "route": "/healthz",
  "status": 200,
  "duration_ms": 0.38
}
$ curl -s 'http://localhost:9290/text-stats-logs/_count?q=tags:_grokparsefailure' | jq -c '{parse_fa
ilures:.count}'
{"parse_failures":0}
$ curl -s http://localhost:9290/text-stats-logs/_search -H 'Content-Type: application/json' -d '{"si
ze":0,"aggs":{"levels":{"terms":{"field":"level.keyword"}},"routes":{"terms":{"field":"route.keywor
d"}}}}' | jq -c '.aggregations | {levels:[.levels.buckets[]|{(.key):.doc_count}],routes:[.routes.buc
kets[]|{(.key):.doc_count}]}'
{"levels":[{"INFO":77563},{"WARNING":8632}],"routes":[{"/healthz":54614},{"/stats":18632},{"/":431
7},{"/ui":4316},{"unmatched":4316}]}
$ docker compose -f observability/compose.yml exec -T app wc -l /var/log/text-stats/app.log; curl -s
localhost:9290/text-stats-logs/_count | jq -c '{indexed_docs:.count}'
86198 /var/log/text-stats/app.log
{"indexed_docs":86198}
$ curl -s localhost:9290/_cluster/health | jq -c '{status,unassigned_shards}'
{"status":"yellow","unassigned_shards":1}
```

Visualization in Kibana

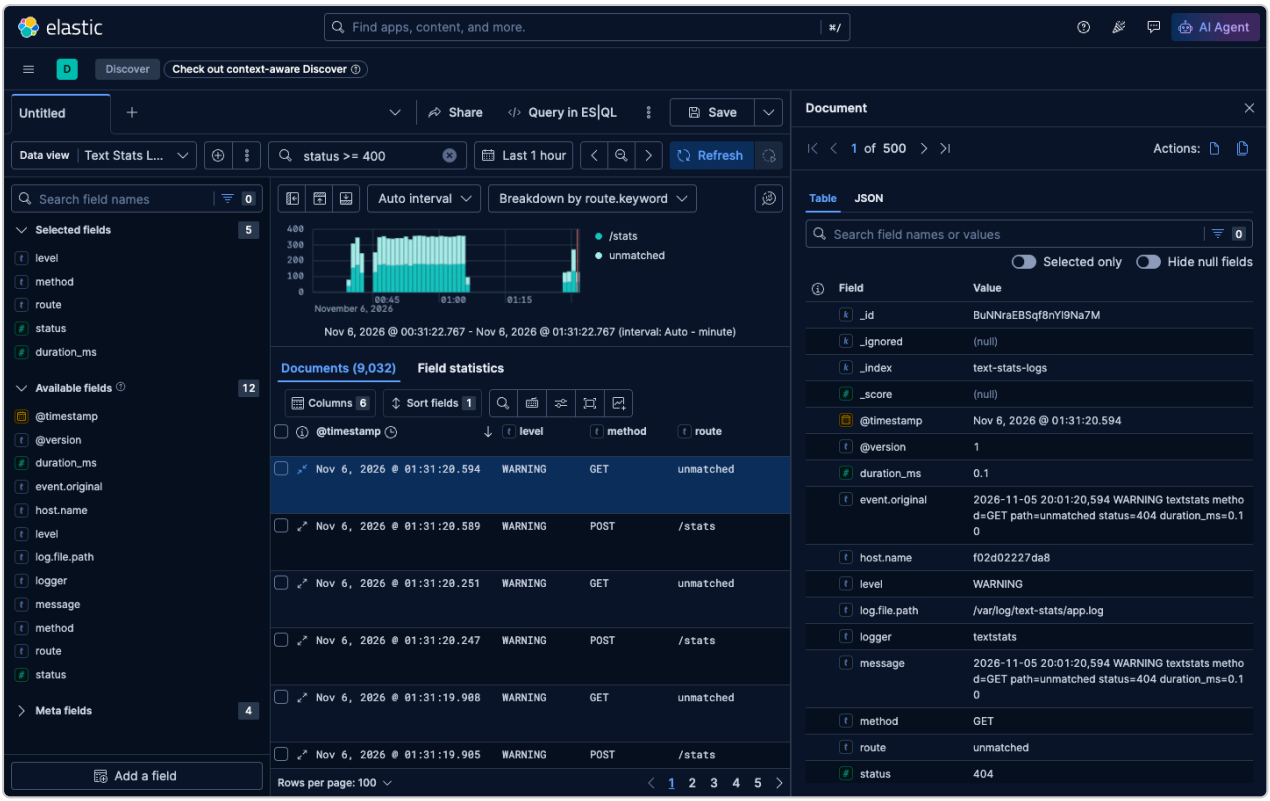
```
$ curl -s -X POST http://localhost:5690/api/data_views/data_view -H 'kbn-xsrf: true' -H 'Content-Typ
e: application/json' -d '{"data_view":{"id":"text-stats","title":"text-stats-logs","name":"Text Stat
s Logs","timeFieldName":"@timestamp"}}' | jq -c '.data_view | {id,title,timeFieldName}'
{"id":"text-stats","title":"text-stats-logs","timeFieldName":"@timestamp"}
```



Discover: parsed documents over the last hour; the two spikes are the Q6 load tests



Query status >= 400, broken down by route: only /stats (400) and unmatched (404)



One document: the raw line (message) next to the fields grok extracted from it

Observations

- **No lost and no unparsed lines:** the log file had 86,198 lines and the index had 86,198 documents, with 0 `_grokparsefailure` tags.
- The level split (77,563 INFO / 8,632 WARNING) matches the traffic: the script sends 2 bad requests in every 6, and the load tests sent only good ones.
- Kibana's time column shows IST and the raw line shows UTC (20:01:20 becomes 01:31:20). Setting `timezone => "UTC"` in the date filter is what makes that correct; without it the events would be shifted.
- Routes come from Flask's URL rule, like the metrics labels, so every unknown URL is grouped as `unmatched`.
- Cluster health turns **yellow** once the index exists. The default 1 replica cannot be placed on a single node. This is expected for a one-node lab cluster.
- Security is disabled in Elasticsearch, so Kibana shows a "your data is not secure" warning. Ports bind to 127.0.0.1 only.